

Q APriori Algorithm - R Pseudocode

Sol:- #APriori Algorithm

```
transactions <- list(  
  C("mobile", "EarPhones"),  
  C("mobile", "charger"),  
  C("EarPhones", "charger"),  
)  
min-support <- 2
```

Step 1: Generate Frequent 1-itemsets

```
L1 <- {}  
Count support of each item  
Keep items with support  $\geq$  min-support
```

```
k <- 2
```

```
while (length(L(k-1)) > 0) {
```

```
  # Candidate Generation
```

```
  Ck <- join(L(k-1), L(k-1))
```

```
  # Pruning
```

```
  Remove Candidates whose subsets are not frequent
```

```
  # Count Support
```

```
  for (each transaction) {
```

```
    Count occurrence of every Candidate
```

```
  }
```

```
# Generate frequent itemsets
```

```
Lk <- "Candidates having supports  $\geq$  min-support"
```

```
k <- k + 1
```

```
}
```

2) FP - Growth Algorithm - Pseudocode

```
# transactions ← list (  
    c("Data science", "Python"),  
    c("Python", "Machine Learning"),  
    c("Data science", "Machine Learning"),  
    c("Python", "Machine Learning"),  
)
```

min - support = 2

Step 1

Count frequency of each book

Step 2

Remove infrequent books

Step 3

Sort books in descending frequency

Step 4

Create FP-Tree

Sort items using frequency order

Insert items into FP-Tree

Update node count

}

Step 5

Mine FP-Tree recursively

if (Tree has single Path){

Generate all combinations

} else {

Create Conditional Pattern base

Build Conditional FP-Tree

Mine recursively

}

Print frequent Pattern

3) Association Rule Generation - R Pseudo Code

Sol:-
frequent_itemsets $\leftarrow \{$
 {Internet},
 {streaming},
 {cloud storage},
 {streaming, cloud storage}
}
Min_Confidence $\leftarrow 0.70$
for (each frequent itemset F) {
 if (length(F) ≥ 2) {
 Generate all non-empty subsets A)
 For (each subset A) {
 B $\leftarrow F - A$
 if (confidence $\geq$ min_confidence) {
 Print Rule A $\rightarrow$ B
 }
 }
 }
}

4) Constraint-Based frequent itemset mining - R Pseudo Code.

Sol:-
transaction $\leftarrow$ list {
 c("Python", "Database");
 c("Python", "AI");
 c("Database", "AI")
 c("Python")
}
Min_Support $\leftarrow 2$
Constraint $\leftarrow$ "Python"
Generate frequent itemsets
Find support of every course.

Keep only itemsets containing "Python"

$k \leftarrow 2$

while (Previous frequent itemsets exist) {

 Generate Candidate itemsets

$k \leftarrow k + 1$

}

Print constrained frequency itemsets

5) Hospital Treatment Hierarchy - R Pseudocode.

sol: transactions $\leftarrow$ list (

 c("Healthcare", "Cardiology", "Egg"),

 c("Healthcare", "Orthopedics"),

 c("Healthcare", "Cardiology"),

 c("Healthcare", "Orthopedics").

)

Min-Support $\leftarrow 2$

Mine frequent treatment itemsets

 Count support of all treatments

 Generate frequent itemsets using Apriori

Hierarchical mining

for (each frequent itemset) {

 if (item belongs to hierarchy) {

 Generalize child item to Parent.

Example !

ECG $\rightarrow$ Cardiology $\rightarrow$ Healthcare

 Update support count

}

}

Print frequent generalized treatment Patterns.